



# Object Oriented Programming

**C++ project on**

# **Bank Management System**

# Presented by :

**Mian Muhammad Jahanzaib**  
**23011519-141**

**Mahnoor Nawaz**  
**23011519-162**

**Zain Haider**  
**23011519-154**

**Salman Butt**  
**23011519-127**

**Areeba Sadia**  
**23011519-148**

# Presented to :

**Mr Zaheer Ahmed**

**Mr Muhammad Noman**

# Introduction :

The program allows users to create accounts with personal details, a secret pin code, a password, and some starting money. Users can then access their accounts using their account number, pin code, and password to withdraw cash, see how much money they have left, or even pay bills electronically. On the other hand, someone with bank administrator privileges can view information about all accounts and even delete accounts or deposit money for users. Overall, this program offers a basic simulation of how a bank management system might function.

# Key Features :-

- **Easy Account Creation** : New users can set up their accounts with just a few clicks, providing basic information and choosing a secure pin and password.
- **Intuitive Navigation** : A clear menu guides users through available options, making it simple to find the features they need, whether it's checking their balance, withdrawing cash, or paying bills.
- **Straightforward Transaction** : Performing actions like withdrawals or bill payments is a smooth process, with clear instructions and easy-to-understand prompts. No more deciphering cryptic menus!
- **Secure Login** : Multiple layers of security, including account numbers, pin codes, and passwords, ensure only authorized users can access their accounts.
- **Comprehensive Management** : For administrators, the program offers features like viewing all accounts, deleting inactive ones, and even depositing funds for users.
- **Data Reliability** : Account information is likely stored securely, allowing users to pick up right where they left off, even if they haven't used the program in a while.

# Header Files :

`<iostream>` : used for input and output operations.

`<string>` : used for string manipulation and operations.

`<vector>` : used for dynamic arrays and containers.

`<fstream>` : used for file input and output operations.

`<memory>` : Mused for dynamic memory management.

`<unordered_map>` : used for associative containers.

`<random>` : used for generating random numbers.

`<algorithm>` : used for various algorithms and functions.

# OOP Concepts :

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

# Encapsulation :

Hiding implementation details and showing only necessary information. Protecting data from external interference.

Example :

Person class attributes are private, and methods are public

# Inheritance :

Creating a new class based on an existing class. Creating a hierarchy of classes.

Example :

Account class inherits from Person class



# Polymorphism :

Method overriding and overloading. Different behaviors for same method name.

Example :

withdraw() method in Account and ATM classes

# Abstraction :

Showing essential features only. Hiding non-essential details.

Example :

Account class abstracting account operations

# Person Class :

## Attributes :

name  
fatherName  
address  
phoneNumber

## Methods :

display()  
getPhoneNumber()

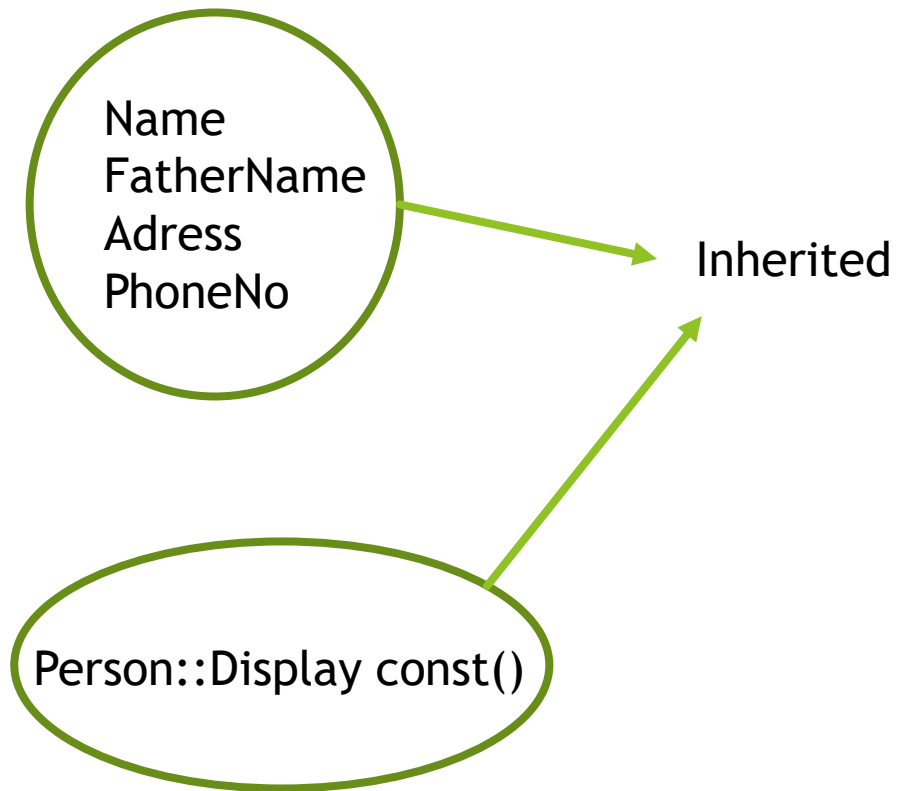
# Account Class (Inherited from Person class)

## Attributes :

- accountNumber
- pinCode
- password
- balance

## Methods :

- authenticate()
- deposit()
- withdraw()
- transfer()
- payBill()



# Bank Class

## Attributes :

- `accounts (unordered_map)` : stores unique pointers to Account objects
- `phoneToAccount (unordered_map)` : maps phone numbers to account numbers

## Methods :

- `saveAccountDetails()` : saves account details to a file
- `loadAccountDetails()` : loads account details from a file
- `generateUniqueAccountNumber()` : generates a unique account number
- `addAccount()` : adds a new account
- `findAccount()` : finds an account by account number
- `findAccountByPhone()` : finds an account by phone number
- `displayAllAccounts()` : displays all accounts
- `deleteAccount()` : deletes an account

# ATM Class

## Attribute :

- bank (reference to Bank object)

## Methods :

- userLogin(): authenticates user and displays balance
- withdrawAmount(): withdraws amount from account
- accountDetails(): displays account details
- goBack(): returns to main menu

# Main Menu

Options :

- New User
- ATM Management
- Bank Management
- Close

# ATM Management Menu

Options :

- User Login
- Withdraw Amount
- Account Details
- Go Back



# Bank Management Menu

Options :

- Deposit
- Withdraw
- Transfer
- Pay Bills
- Delete User Record
- Show All Records
- Go Back

# Code/Input

```
#include <iostream>
#include <string>
#include <vector>
#include <fstream>
#include <memory>
#include <unordered_map>
#include <random>
#include <algorithm>
```

```
using namespace std;
```

```
class Account;
class Bank;
class ATM;
```

```
class Person {
protected:
    string name;
    string fatherName;
    string address;
    string phoneNumber;
```

```
public:
    Person(const string& name, const string& fatherName,
const string& address, const string& phoneNumber)
        : name(name), fatherName(fatherName),
address(address), phoneNumber(phoneNumber) {}

    virtual ~Person() = default;

    virtual void display() const {
        cout << "Name: " << name << endl;
        cout << "Father's Name: " << fatherName << endl;
        cout << "Address: " << address << endl;
        cout << "Phone Number: " << phoneNumber << endl;
    }

    string getPhoneNumber() const {
        return phoneNumber;
    }
};
```

```
class Account : public Person {
private:
    string accountNumber;
    string pinCode;
    string password;
    double balance;
    unordered_map<string, bool> paidBills;
public:
    Account(const string& name, const string& fatherName, const string&
address, const string& phoneNumber, const string& accountNumber, const
string& pinCode, const string& password, double initialBalance)
        : Person(name, fatherName, address, phoneNumber),
accountNumber(accountNumber), pinCode(pinCode), password(password),
balance(initialBalance) {}
    ~Account() {
}

    bool authenticate(const string& enteredPinCode, const string&
enteredPassword) const {
        return pinCode == enteredPinCode && password == enteredPassword;
    }
    bool authenticatePin(const string& enteredPinCode) const {
        return pinCode == enteredPinCode;
    }
}
```

```
bool authenticatePassword(const string& enteredPassword)
const {
    return password == enteredPassword;
}
void deposit(double amount) {
    balance += amount;
}
bool withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
        return true;
    }
    return false;
}
bool transfer(Account& toAccount, double amount) {
    if (withdraw(amount)) {
        toAccount.deposit(amount);
        return true;
    }
}
return false;
}
```

```
void payBill(const string& billNumber, double amount) {
    if (paidBills.find(billNumber) == paidBills.end()) {
        balance -= amount;
        paidBills[billNumber] = true;
    }
}

bool isBillPaid(const string& billNumber) const {
    return paidBills.find(billNumber) != paidBills.end();
}

double getBalance() const {
    return balance;
}

void display() const override {
    Person::display();
    cout << "Account Number: " << accountNumber << endl;
    cout << "Balance: " << balance << endl;
}

string getPassword() const {
    return password;
}

string getPinCode() const {
    return pinCode;
}

string getAccountNumber() const {
    return accountNumber;}}};
```

```
class Bank {
private:
    unordered_map<string, unique_ptr<Account>> accounts;
    unordered_map<string, string> phoneToAccount;
    void saveAccountDetails() {
        ofstream file("accounts.txt");
        if (file.is_open()) {
            for (const auto& pair : accounts) {
                const Account& account = *pair.second;
                file << account.getAccountNumber() << " " <<
account.getBalance() << " " << account.getPinCode() << " " <<
account.getPassword() << endl;
            }
            file.close();
        }
    }
}
```

```
void loadAccountDetails() {
    ifstream file("accounts.txt");
    if (file.is_open()) {
        string accountNumber;
        double balance;
        string pinCode;
        string password;
        while (file >> accountNumber >> balance >>
pinCode >> password) {
            accounts[accountNumber] =
make_unique<Account>("Loaded", "Loaded", "Loaded",
"Loaded", accountNumber, pinCode, password, balance);
        }
        file.close();
    }
}
```



```
string generateUniqueAccountNumber() {
    const string characters = "abcdefghijklmnopqrstuvwxyz0123456789";
    const int length = 5;
    string accountNumber;
    random_device rd;
    mt19937 generator(rd());
    uniform_int_distribution<> distribution(0, characters.size() -
1);

    do {
        accountNumber.clear();
        for (int i = 0; i < length; ++i) {
            accountNumber += characters[distribution(generator)];
        }
    } while (accounts.find(accountNumber) != accounts.end());

    return accountNumber;
}

public:
    Bank() {
        loadAccountDetails();
    }
```

```
~Bank() {
    saveAccountDetails();
}

string addAccount(const string& name, const string& fatherName,
const string& address, const string& phoneNumber, const string&
pinCode, const string& password, double initialBalance) {
    string accountNumber = generateUniqueAccountNumber();
    auto account = make_unique<Account>(name, fatherName, address,
phoneNumber, accountNumber, pinCode, password, initialBalance);
    accounts[accountNumber] = move(account);
    phoneToAccount[phoneNumber] = accountNumber;
    return accountNumber;
}

Account* findAccount(const string& accountNumber) {
    if (accounts.find(accountNumber) != accounts.end()) {
        return accounts[accountNumber].get();
    }
    return nullptr;
}

string findAccountByPhone(const string& phoneNumber) {
    if (phoneToAccount.find(phoneNumber) != phoneToAccount.end()) {
        return phoneToAccount[phoneNumber];
    }
    return "";}
}
```

```
void displayAllAccounts() const {
    if (accounts.empty()) {
        cout << "No records available." << endl;
    }
    else {
        for (const auto& pair : accounts) {
            pair.second->display();
        }
    }
}

void deleteAccount(const string& accountNumber) {
    if (accounts.find(accountNumber) !=
accounts.end()) {
        phoneToAccount.erase(accounts[accountNumber]-
>getPhoneNumber());
        accounts.erase(accountNumber);
    }
}

};
```

```
class ATM {
private:
    Bank& bank;
public:
    ATM(Bank& bank) : bank(bank) {}
    void userLogin() {
        string accountNumber;
        string pinCode, password;
        for (int attempt = 0; attempt < 3; ++attempt) {
            cout << "Enter Account Number: ";
            cin >> accountNumber;
            cout << "Enter Pin Code: ";
            cin >> pinCode;
            cout << "Enter Password: ";
            cin >> password;
            Account* account = bank.findAccount(accountNumber);
            if (account && account->authenticate(pinCode, password)) {
                cout << "Login successful. Your balance is: " << account-
>getBalance() << endl;
                return;
            }
            cout << "Invalid credentials. Try again." << endl;
        }
        cout << "Too many failed attempts. Returning to main menu." << endl;}
}
```

```
void withdrawAmount() {
    string accountNumber;
    string pinCode;
    double amount;
    cout << "Enter Account Number: ";
    cin >> accountNumber;
    cout << "Enter Pin Code: ";
    cin >> pinCode;
    Account* account = bank.findAccount(accountNumber);
    if (account && account->authenticatePin(pinCode)) {
        cout << "Enter amount to withdraw: ";
        cin >> amount;
        if (account->withdraw(amount)) {
            cout << "Withdrawal successful. Your new balance is: " <<
account->getBalance() << endl;
        }
        else {
            cout << "Insufficient funds." << endl;
        }
    }
    else {
        cout << "Invalid pin code or account number." << endl;
    }
}
```

```
void accountDetails() {
    string accountNumber;
    string password;

    cout << "Enter Account Number: ";
    cin >> accountNumber;
    cout << "Enter Password: ";
    cin >> password;

    Account* account = bank.findAccount(accountNumber);
    if (account && account->authenticatePassword(password)) {
        account->display();
    }
    else {
        cout << "Invalid credentials." << endl;
    }
}

void goBack() {
    cout << "Returning to main menu." << endl;
}

};
```

```
int main() {
    Bank bank;
    ATM atm(bank);
    int choice;

    while (true) {
        cout << "\nMain Menu:\n";
        cout << "1. New User\n";
        cout << "2. ATM Management\n";
        cout << "3. Bank Management\n";
        cout << "4. Close\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
```

```
case 1: {
    string name, fatherName, address, phoneNumber, pinCode, password;
    double initialBalance;
    cout << "Enter Name: ";
    cin >> name;
    cout << "Enter Father's Name: ";
    cin >> fatherName;
    cout << "Enter Address: ";
    cin >> address;
    for (int attempt = 0; attempt < 3; ++attempt) {
        cout << "Enter 11-Digit Phone Number: ";
        cin >> phoneNumber;
        if (phoneNumber.length() == 11 && all_of(phoneNumber.begin(),
phoneNumber.end(), ::isdigit)) {
            if (bank.findAccountByPhone
(phoneNumber).empty()) {
                break;
            }
            else {
                cout << "Phone number already in use. Try again." <<
endl;
            }
        }
        else {
            cout << "Invalid phone number. Try again." << endl;}}}
```



```
if (phoneNumber.length() != 11) {
    cout << "Too many failed attempts. Returning to main menu." <<
endl;
    break;
}
while (true) {
    cout << "Enter 4-Digit Pin Code: ";
    cin >> pinCode;
    if (pinCode.length() == 4 && all_of(pinCode.begin(),
pinCode.end(), ::isdigit)) {
        break;
    }
    else {
        cout << "Invalid pin code. Try again." << endl;
    }
}
while (true) {
    cout << "Enter 8-Character Password (letters and/or digits): ";
    cin >> password;
    if (password.length() == 8 && all_of(password.begin(),
password.end(), [](char c) { return isalnum(c); }))) {
        break;
    }
    else {
        cout << "Invalid password. Try again." << endl;}}
```

```
cout << "Enter Initial Balance: ";
cin >> initialBalance;

string accountNumber = bank.addAccount(name, fatherName,
address, phone Number, pinCode, password, initialBalance);
cout << "Account created successfully. Your account number is: "
<< accountNumber << endl;
break;
}
case 2:
while (true) {
cout << "\nATM Management Menu:\n";
cout << "1. User Login\n";
cout << "2. Withdraw Amount\n";
cout << "3. Account Details\n";
cout << "4. Go Back\n";
cout << "Enter your choice: ";
cin >> choice;
switch (choice) {
case 1:
atm.userLogin();
break;
```

```
case 2:
    atm.withdrawAmount();
    break;
case 3:
    atm.accountDetails();
    break;
case 4:
    atm.goBack();
    goto MAIN_MENU;
default:
    cout << "Invalid choice. Try again." << endl;
}
}
case 3:
    while (true) {
        cout << "\nBank Management Menu:\n";
        cout << "1. Deposit\n";
        cout << "2. Withdraw\n";
        cout << "3. Transfer\n";
        cout << "4. Pay Bills\n";
        cout << "5. Delete User Record\n";
        cout << "6. Show All Records\n";
        cout << "7. Go Back\n";
        cout << "Enter your choice: ";
        cin >> choice;
```

```
switch (choice) {
case 2: {
    string accountNumber;
    double amount;
    cout << "Enter Account Number: ";
    cin >> accountNumber;
    cout << "Enter Amount to Withdraw: ";
    cin >> amount;

    Account* account = bank.findAccount(accountNumber);
    if (account) {
        if (account->withdraw(amount)) {
            cout << "Withdrawal successful. Your new balance is: " <<
account->getBalance() << endl;
        }
        else {
            cout << "Insufficient funds." << endl;
        }
    }
    else {
        cout << "Account not found." << endl;
    }
    break;
}
```

```
case 3: {
    string fromAccountNumber, toAccountNumber;
    double amount;
    cout << "Enter Your Account Number: ";
    cin >> fromAccountNumber;
    cout << "Enter Receiver's Account Number: ";
    cin >> toAccountNumber;
    cout << "Enter Amount to Transfer: ";
    cin >> amount;
    Account* fromAccount =
bank.findAccount(fromAccountNumber);
    Account* toAccount = bank.findAccount(toAccountNumber);
    if (!fromAccount && !toAccount) {
        cout << "Both your account number and the receiver's
account number are incorrect." << endl;
    }
}
```

```
    else if (!fromAccount) {
        cout << "Your account number is
incorrect." << endl;
    }
    else if (!toAccount) {
        cout << "Receiver's account number
not found." << endl;
    }
    else if (fromAccount->transfer(*toAccount, amount)) {
        cout << "Transfer successful." <<
endl;
    }
    else {
        cout << "Insufficient funds." <<
endl;
    }
    break;}
```

```
case 4: {  
    string accountNumber, billNumber;  
    int billType;  
    double amount;  
  
    cout << "Enter Account Number: ";  
    cin >> accountNumber;  
    cout << "Enter Bill Type (1 for  
Electricity, 2 for Gas, 3 for Internet): ";  
    cin >> billType;  
    cout << "Enter 10-Digit Bill Number: ";  
    cin >> billNumber;  
    cout << "Enter Bill Amount: ";  
    cin >> amount;
```

```
Account* account = bank.findAccount(accountNumber);
    if (account && billNumber.length() == 10) {
        if (account->isBillPaid(billNumber)) {
            cout << "Bill already paid." << endl;
        }
        else if (account->getBalance() >= amount) {
            account->payBill(billNumber, amount);
            cout << "Bill paid successfully. Your new balance is:
" << account->getBalance() << endl;
        }
        else {
            cout << "Insufficient funds." << endl;
        }
    }
    else {
        cout << "Invalid bill number or account not found." <<
endl;
    }
    break;
}
```



```
case 5: {
    string accountNumber;
    cout << "Enter Account Number: ";
    cin >> accountNumber;

    bank.deleteAccount(accountNumber);
    cout << "Account deleted successfully." << endl;
    break;
}
case 6:
    bank.displayAllAccounts();
    break;
case 7:
    goto MAIN_MENU;
default:
    cout << "Invalid choice. Try again." << endl;
}
}
case 4:
    cout << "Exiting the system." << endl;
    return 0;
default:
    cout << "Invalid choice. Try again." << endl;
}
```

```
MAIN_MENU:  
    continue;  
}  
  
return 0;  
}
```

# Output

## Main Menu

```
----- Bank Management System -----
```

```
Main Menu :-
```

- (1) New User
- (2) ATM Management
- (3) Bank Management
- (4) Close

```
Enter your choice :
```

# New User Functionality

```
Enter your choice : 1  
  
Enter Name : Mian Jahanzaib  
Enter Father's Name : Babar Shahzad  
Enter Address : Wasu  
Enter 11-Digit Phone Number : 03000000000  
Set 4-Digit Pin Code : 1122  
Enter 8-Character Password (letters and/or digits) : passpass  
Enter Initial Balance : 10000  
  
-----> Account created successfull <-----  
Your account number is : qu5vk
```

## Atm Menu

ATM Management Menu :-

- (1) User Login
- (2) Withdraw Amount
- (3) Account Details
- (4) Go Back

Enter your choice :

## User login Functionality (Atm menu)

```
Enter your choice : 1  
Enter Account Number : qu5vk  
Enter Pin Code : 1122  
Enter Password : passpass  
  
-----> Login successful <-----  
Your balance is : 10000
```

## Withdraw Amount Functionality (Atm Menu)

```
Enter your choice : 2  
Enter Account Number : qu5vk  
Enter Pin Code : 1122  
Enter amount to withdraw : 34  
  
-----> Withdrawal successful <-----  
Your new balance is : 9966
```

## Show Account Details Functionality (Atm menu)

```
Enter your choice : 3  
  
Enter Account Number : qu5vk  
Enter Password : passpass  
:  
Name : Mian Jahanzaib  
Father's Name : Babar Shahzad  
Address : Wasu  
Phone Number : 03000000000  
  
Account Number : qu5vk  
Balance : 9966
```



## Go back to Main menu Functionality (Atm menu)

```
Enter your choice : 4  
Returning to main menu.
```

```
----- Bank Management System -----
```

```
Main Menu :-
```

- (1) New User
- (2) ATM Management
- (3) Bank Management
- (4) Close

```
Enter your choice :
```

# Bank Menu

Main Menu :-

- (1) New User
- (2) ATM Management
- (3) Bank Management
- (4) Close

Enter your choice : 3

Bank Management Menu :-

- (1) Deposit
- (2) Withdraw
- (3) Transfer
- (4) Pay Bills
- (5) Delete User Record
- (6) Show All Records
- (7) Go Back

Enter your choice :

## Deposit Functionality (Bank Menu)

```
Enter your choice : 1  
Enter Account Number : qu5vk  
Enter Amount to Deposit : 3562  
  
-----> Deposit successful <-----  
Your new balance is : 13528
```

## Withdraw Functionality (Bank Menu)

```
Enter your choice : 2  
Enter Account Number : qu5vk  
Enter Amount to Withdraw : 232  
  
-----> Withdrawal successful <-----  
Your new balance is : 13296
```

## Transfer Amount Functionality (Bank Menu)

```
Enter your choice : 3
```

```
Enter Your Account Number : qu5vk
```

```
Enter Receiver's Account Number : av526
```

```
Enter Amount to Transfer : 123
```

```
-----> Transfer successful <-----
```

## Pay Bill Functionality (Bank Menu)

```
Enter your choice : 4
```

```
Enter Account Number : qu5vk
```

```
Enter Bill Type ( 1 for Electricity, 2 for Gas, 3 for Internet) : 1
```

```
Enter 10-Digit Bill Number : 5678317723
```

```
Enter Bill Amount : 1423
```

```
-----> Bill paid successfully <-----
```

```
Your new balance is : 11750
```

## Delete user account Functionality (Bank Menu)

```
Enter your choice : 5
```

```
Enter Account Number : av526
```

```
-----> Account deleted successfully <-----
```

## Show all records Functionality (Bank Menu)

Enter your choice : 6

Name : Mian Jahanzaib  
Father's Name : Babar Shahzad  
Address : Wasu  
Phone Number : 030000000000  
Account Number : qu5vk  
Balance : 11750



## Go back to main menu Functionality (Bank Menu)

```
Enter your choice : 7
```

```
----- Bank Management System -----
```

```
Main Menu :-
```

- (1) New User
- (2) ATM Management
- (3) Bank Management
- (4) Close

```
Enter your choice :
```

The background features abstract, overlapping geometric shapes in various shades of green, primarily on the right side of the frame. These shapes include triangles and polygons, some of which are semi-transparent, creating a layered effect. The overall aesthetic is modern and clean.

# Thank You!